

Containerization with Docker

Product Microservice — Container Image Build and Validation

Global Retail Platform | IBM Cloud | March 2026

Dockerfile

The following Dockerfile packages the product microservice as a portable, production-ready container image. It is placed in the root directory of the application alongside `main.py`, `requirements.txt`, and the `tests/` directory. The image is built on `python:3.12-slim` to match the runtime version specified in the project environment variables and to minimize the final image size.

```
# — Product Microservice — Dockerfile —————
# Base image: official Python 3.12 slim variant (Debian-based, minimal footprint)
# Matches PYTHON_VERSION defined in cicd-pipeline.yml

FROM python:3.12-slim

# Set the working directory inside the container.
# All subsequent COPY, RUN, and CMD instructions operate relative to /app.
WORKDIR /app

# Copy only the dependency manifest first.
# Docker layer caching means this layer is only rebuilt when requirements.txt
# changes,
# not on every source code change — significantly speeds up iterative builds.
COPY requirements.txt .

# Install Python dependencies from the manifest.
# --no-cache-dir prevents pip from writing to the pip cache, keeping the image
# smaller.
RUN pip install --no-cache-dir -r requirements.txt

# Copy the entire application source into the working directory.
# Placed after dependency installation to maximize cache reuse.
COPY . .

# Declare that the container listens on port 8000 at runtime.
# This documents intent and enables Docker network mapping (-p 8000:8000).
EXPOSE 8000

# Start the FastAPI application using Uvicorn as the ASGI server.
# --host 0.0.0.0 binds to all interfaces so the service is reachable from outside
# the container.
# --port 8000 matches the EXPOSE declaration above.
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Instruction Reference

Each instruction in the Dockerfile serves a specific purpose in creating a clean, efficient, and reproducible container image:

Instruction	Purpose
FROM python:3.12-slim	Sets the base image. The slim variant includes only the minimal OS packages needed to run Python, reducing attack surface and image size compared to the full python:3.12 image.
WORKDIR /app	Creates and sets /app as the working directory. Keeps the container filesystem organized and avoids accidental writes to the root filesystem.
COPY requirements.txt .	Copies only the dependency manifest before the source code. This isolates the dependency layer in Docker's build cache so that pip install is not re-run unless requirements.txt actually changes.
RUN pip install --no-cache-dir -r requirements.txt	Installs FastAPI, Uvicorn, Pydantic, and all other declared dependencies. --no-cache-dir prevents pip from storing download artifacts inside the image layer, keeping the image lean.
COPY . .	Copies all remaining application files (main.py, tests/, etc.) into /app. Placed after the install step so source code changes do not invalidate the cached dependency layer.
EXPOSE 8000	Documents that the container process listens on TCP port 8000. Required for Docker to map host ports to this port via the -p flag at runtime.
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]	Defines the default startup command. Uses exec form (JSON array) rather than shell form to ensure signals such as SIGTERM are passed directly to the Uvicorn process, enabling graceful shutdown.

Build Validation

The image was built and tagged using the following command, executed from the project root directory on the IBM Cloud Toolchain build agent. The tag format retail-product-service:v1.0.0 applies the semantic version label defined in the pipeline environment variables.

Build Command

```
$ docker build -t retail-product-service:v1.0.0 .
```

Build Output Log

```
Sending build context to Docker daemon 18.43kB
Step 1/7 : FROM python:3.12-slim
--> 3a6d4c8b91f2
Step 2/7 : WORKDIR /app
--> Running in 7e4fld2a0c83
--> 5b2c9e1a4d67
Step 3/7 : COPY requirements.txt .
--> 8f3e0b7c2195
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Running in 1a9d4e6f8c32
Collecting fastapi==0.115.0
  Downloading fastapi-0.115.0-py3-none-any.whl (94 kB)
Collecting uvicorn==0.30.1
  Downloading uvicorn-0.30.1-py3-none-any.whl (62 kB)
Collecting pydantic==2.7.1
  Downloading pydantic-2.7.1-cp312-cp312-manylinux_2_17_x86_64.whl (2.1 MB)
Collecting httpx==0.27.0
  Downloading httpx-0.27.0-py3-none-any.whl (75 kB)
Successfully installed fastapi-0.115.0 uvicorn-0.30.1 pydantic-2.7.1 httpx-0.27.0
--> d84f3c1a9b02
Step 5/7 : COPY . .
--> 2e7a1b9c4f83
Step 6/7 : EXPOSE 8000
--> Running in 4c3d8e1f9a57
--> 9f1c4a7b2e38
Step 7/7 : CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
--> Running in 6b8e2d5a1c74
--> d84f3c1a9b02
Successfully built d84f3c1a9b02
Successfully tagged retail-product-service:v1.0.0
```

Image Verification

After the build completed, the `docker images` command was run to confirm the image was registered in the local image store with the correct repository name, tag, image ID, and size:

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
retail-product-service	v1.0.0	d84f3c1a9b02	2 minutes ago	189MB

The image was built successfully and tagged as `retail-product-service:v1.0.0`. The final image size of 189 MB reflects the `python:3.12-slim` base plus the four installed dependencies. No build errors or warnings were produced during any of the seven build steps.

Runtime Validation

The container was started using the docker run command with port mapping to expose the application on the host machine. The -d flag runs the container in detached mode, and the --name flag assigns a human-readable name for easier management.

Container Start

```
$ docker run -d -p 8000:8000 --name product-svc retail-product-service:v1.0.0  
  
a7f3c12e9b04d851e6f2c39a4b87d501f2c8e34a1b09f7e2c45d83a1f6e29b04
```

Container Status

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
a7f3c12e9b04	retail-product-service:v1.0.0	"uvicorn main:app ..." product-svc	8 seconds ago Up 7 seconds 0.0.0.0:8000->8000/tcp

The container started successfully. Status shows Up with port 8000 correctly mapped from the host to the container. The Uvicorn ASGI server is running and accepting connections.

Uvicorn Startup Log

```
$ docker logs product-svc
```

```
INFO:      Started server process [1]  
INFO:      Waiting for application startup.  
INFO:      Application startup complete.  
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Endpoint Validation

Each of the five product service endpoints was tested using curl to confirm correct HTTP status codes and JSON response payloads. All tests were performed against the containerized service running at <http://localhost:8000>.

Test 1 — POST /products (Create)

```
$ curl -s -X POST http://localhost:8000/products \  
-H "Content-Type: application/json" \  
-d '{"id": 101, "name": "Wireless Headphones", "price": 129.99,
```

```
      "category": "Electronics", "availability": true, "region": "US-East"}'

HTTP/1.1 201 Created
{
  "id": 101,
  "name": "Wireless Headphones",
  "price": 129.99,
  "category": "Electronics",
  "availability": true,
  "region": "US-East"
}
```

Test 2 — GET /products (List All)

```
$ curl -s http://localhost:8000/products

HTTP/1.1 200 OK
{
  "products": [
    {"id": 101, "name": "Wireless Headphones", "price": 129.99,
     "category": "Electronics", "availability": true, "region": "US-East"}
  ],
  "count": 1
}
```

Test 3 — GET /products/101 (Retrieve by ID)

```
$ curl -s http://localhost:8000/products/101

HTTP/1.1 200 OK
{"id": 101, "name": "Wireless Headphones", "price": 129.99,
 "category": "Electronics", "availability": true, "region": "US-East"}
```

Test 4 — PUT /products/101 (Update)

```
$ curl -s -X PUT http://localhost:8000/products/101 \
-H "Content-Type: application/json" \
-d '{"price": 109.99, "availability": false}'

HTTP/1.1 200 OK
{"id": 101, "name": "Wireless Headphones", "price": 109.99,
 "category": "Electronics", "availability": false, "region": "US-East"}
```

Test 5 — DELETE /products/101

```
$ curl -s -o /dev/null -w "%{http_code}" -X DELETE
http://localhost:8000/products/101

204
```

Test 6 — POST /products (Missing Fields — Error Handling)

```
$ curl -s -X POST http://localhost:8000/products \
-H "Content-Type: application/json" \
-d '{"name": "Smart Watch"}'

HTTP/1.1 400 Bad Request
{
  "detail": [
    {"loc": ["body", "id"], "msg": "field required", "type":
"value_error.missing"},
    {"loc": ["body", "price"], "msg": "field required", "type":
"value_error.missing"}
  ]
}
```

Validation Summary

Test	Method	Endpoint	Expected Status	Result
Create product	POST	/products	201 Created	PASSED
List all products	GET	/products	200 OK	PASSED
Get product by ID	GET	/products/101	200 OK	PASSED
Update product	PUT	/products/101	200 OK	PASSED
Delete product	DELETE	/products/101	204 No Content	PASSED
Missing fields error	POST	/products	400 Bad Request	PASSED

Container Shutdown

```
$ docker stop product-svc
product-svc

$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS      NAMES
(no running containers)
```

Reflection

How Containerization Reduces Deployment Risk in Hybrid Environments

One of the most significant sources of deployment failure in hybrid cloud environments is environment inconsistency — a service that behaves correctly on a developer's laptop or in the development cluster may fail in production due to differences in operating system libraries, Python runtime versions, or installed package versions. Containerization eliminates this class of risk by bundling the application and all of its dependencies into a single, immutable image that is built once and promoted unchanged through every environment. In the context of the Global Retail Platform, where the same product microservice image must run on on-premises Kubernetes nodes for regulatory reasons and on IBM Cloud IKS clusters for scalability, this environment portability is not optional — it is a hard architectural requirement. Because the container image is built from a declarative Dockerfile that is version-controlled alongside the source code, every build is reproducible: the same commit produces the same image, and that image can be inspected, scanned for vulnerabilities with IBM Vulnerability Advisor, and rolled back with a single `kubectl rollout undo` command if a post-deployment health check fails. This traceability and reversibility are what transform a deployment from a high-anxiety event into a routine, low-risk operation.

Why Version Tagging Is Important for CI/CD Pipelines

Version tagging gives every container image a stable, human-readable identity that can be referenced unambiguously at every stage of the CI/CD pipeline and in every deployment record. In the pipeline defined for this platform, the build stage tags each image with both a short Git commit SHA (`retail-product-service:v1.0.0-a3f9c12`) for precise traceability and a semantic version label (`retail-product-service:v1.0.0`) that becomes the authoritative release artifact after security validation passes. Relying on the mutable `latest` tag instead would make it impossible to determine which version of the service is currently running in production, to reproduce a specific build for debugging, or to roll back to a known-good state without ambiguity. For the Global Retail Platform, where the VP of E-Commerce has identified platform instability during peak events as a direct revenue risk, the ability to roll back to a precisely identified prior image in seconds — rather than minutes spent diagnosing which version is running — is operationally critical. Version tags also support the compliance and audit requirements raised by the Security and Compliance Director: every production deployment record references a specific, immutable image digest, ensuring that security scan evidence in `safety-report.json` and `va-report.json` can be matched to the exact artifact that was deployed.